

Stack Offline-First Profesional

- ✓ 100% local — Cero internet requerido
- ✓ Sin servidores — Sin CDNs — Sin builds
- ✓ Datos del cliente siempre en su maquina
- ✓ Código fuente portable — .html o .exe
- ✓ Español / Moneda LatAm — Soberanía digital

1. Filosofia del Stack

Este stack esta disenado para un tipo especifico de desarrollador: el profesional freelance que entrega apps de escritorio a clientes en Latinoamerica, donde el acceso a internet puede ser limitado y la privacidad de datos es critica. No es para SaaS, no es para la nube, no es para startups que buscan escalar a millones.

Principios inquebrantables

Principio	Implicacion	Que queda prohibido
Cero Internet	La app funciona sin conectividad. Todas las dependencias son locales.	CDNs, Google Fonts, fetch a URLs externas, telemetria
Sin servidor	El usuario abre la app con doble clic en index.html o ejecuta un .exe.	Node.js, Express, bases de datos externas, Docker, APIs REST remotas
Sin build	Editas el codigo, refrescas el navegador o compilas solo para entrega final.	Webpack, Vite, Babel, TypeScript compiler en dev loop
Localizacion LatAm	Todo en espanol, formato DD/MM/YYYY, moneda \$1.234,56	Mensajes en ingles, formato ISO, fecha MM/DD/YYYY
Privacidad	Cero analytics, cero telemetria, cero envio de datos	Google Analytics, Sentry, cualquier servicio de terceros

Audiencia objetivo: Desarrolladores freelance que entregan software de gestion, facturacion, inventarios, CRM, ERPs o herramientas internas a clientes que valoran la autonomia y la privacidad de sus datos. El producto final es un archivo o ejecutable que el cliente guarda en su escritorio.

2. Analisis: Offline-First vs ATEJE

Ambos stacks comparten filosofia pero difieren en implementacion. ATEJE introduce servidor (Bun) y compilacion a cambio de SQLite y capacidades IA. La tabla muestra donde convergen y donde divergen.

Aspecto	Offline-First	ATEJE	Unificado
Runtime	file:// (doble clic)	Bun (servidor)	file:// + Bun opcional
BD Local	Dexie (IndexedDB)	bun:sqlite (SQLite)	Dexie + bun:sqlite
Crypto	CryptoJS	Web Crypto API	Ambos (deteccion)
CSS	CDN play (sin build)	Tailwind CLI (compilado)	CDN play + CLI opcional
Iconos	Bootstrap Icons	Lucide	Lucide (default)
Build	Ninguno	bun build --compile	Opcional solo prod
Peso .exe	N/A (solo .html)	50-55 MB	50-55 MB (si se compila)
IA Local	No incluida	Transformers.js + ONNX	Opcional (skill IA)
Librerias extra	ApexCharts, jsPDF, SheetJS, Pako, Animate.css	No incluidas	Todas disponibles
Proteccion IP	Codigo visible	Binario ofuscado	Binario al compilar

Conclusion: El stack unificado toma lo mejor de ambos. Usa Dexie para portabilidad (funciona siempre, incluso en file://) y ofrece bun:sqlite como upgrade cuando se detecta Bun runtime. Esto permite que el mismo codigo base se ejecute en 3 modos.

3. Stack Unificado Recomendado

La propuesta de convergencia: un solo stack que funciona en 3 modos segun el contexto. El desarrollador escribe una vez, el runtime decide que capacidades activar.

Modo	Comando	BD	Crypto	CSS	Uso
Lite	Doble clic index.html	Dexie (IndexedDB)	CryptoJS	Tailwind CDN play	Desarrollo rapido, demos
Dev	bun --hot servidor.ts	Dexie + bun:sqlite	CryptoJS + Web Crypto	Tailwind CDN play	Desarrollo con backend local
Prod	./app.exe	bun:sqlite (SQLite)	Web Crypto API	Tailwind CLI (minified)	Entrega a cliente final

3.1 Layout del proyecto generado

```
mi-app/
index.html # Entry point (Modo Lite: doble clic aqui)
servidor.ts # Servidor Bun (Modo Dev/Prod)
tailwind.config.js # Config DaisyUI + custom colors
assets/ # Librerias JS locales (sin CDN)
alpine.min.js
dexie.min.js
crypto-js.min.js
tailwind.min.css # CDN play descargado local
lucide.umd.min.js
apexcharts.min.js # Carga diferida bajo demanda
jspdf.umd.min.js # Carga diferida bajo demanda
xlsx.full.min.js # Carga diferida bajo demanda
src/
core/
app.js # Alpine store global, router
db.js # Capa de datos abstraída
crypto-helpers.js # CryptoJS + Web Crypto wrapper
config.js # APP_CONFIG desde project.config.js
modules/
dashboard/
clientes/
productos/
... # Un modulo por funcionalidad
models/ # (Opcional) Modelos ONNX para IA local
project.config.js # Config white-label
```

3.2 Deteccion de runtime en codigo

La capa de datos (db.js) y crypto (crypto-helpers.js) detectan el runtime automaticamente para usar Dexie o SQLite, CryptoJS o Web Crypto segun corresponda. Esto permite que el mismo modulo funcione en los 3 modos sin cambios.

```
// db.js - Capa de datos abstracta\nconst tieneBun = typeof Bun !== 'undefined';\n\nexport const db = {\n  async getAll(tabla) {\n    if (tieneBun) return fetch(`/api/${tabla}`).then(r => r.json());\n    return dbLocal[tabla].toArray();\n  },\n  async save(tabla, datos) {\n    if (tieneBun) return fetch(`/api/${tabla}`, { method: 'POST', body: JSON.stringify(datos) });\n    return dbLocal[tabla].put(datos);\n  }\n};
```

4. Catalogo de Tecnologias Compatibles

Tecnologias adicionales que mantienen la filosofia offline-first. Cada una evaluada por compatibilidad con file://, peso, licencia y caso de uso.

4.1 Busqueda y texto

Tecnologia	Proposito	Peso	file://	Licencia
FlexSearch	Busqueda full-text ultra rapida con indexacion persistente. 1000x mas rapido que Fuse.js en datasets grandes.	4-16 KB	Si	Apache-2.0
MiniSearch	TF-IDF + BM25, fuzzy, sugerencias. Soporta anadir/eliminar documentos en caliente.	~7 KB	Si	MIT
Fuse.js	Fuzzy matching simple. Ideal para autocomplete y busqueda tolerante a errores.	~8 KB	Si	Apache-2.0
Lunr.js	TF-IDF classico estilo Solr. No permite updates parciales.	~8 KB	Si	MIT

4.2 Bases de datos locales

Tecnologia	Proposito	Peso	file://	Licencia
LokiJS	NoSQL en memoria. Persistencia via IndexedDB. 1.1M ops/s. Rapido pero sin mantenimiento desde 2021.	262 KB	Si	MIT
SQL.js	SQLite completo via WASM. JOINS, subconsultas, indices. Requiere servidor para WASM.	690 KB	Parcial	MIT
DuckDB-WASM	OLAP analitico. Procesa CSV, Parquet, JSON. Ideal para reportes y dashboard.	3.2 MB	Parcial	MIT

4.3 Empaquetado para escritorio

Opcion	Peso .exe	Runtime incluido	Complejidad	Ideal para
Bun --compile	50-55 MB	Bun (JS runtime completo)	Baja (JS puro)	Apps que necesitan backend local completo
Neutralino.js	1-2 MB	WebView del OS	Media (config)	Apps ligeras, wrappers de UI
Tauri	5-15 MB	WebView OS + Rust backend	Media-Alta (Rust basico)	Apps medianas con rendimiento
Electron	150-300 MB	Chromium completo	Baja (JS puro)	Apps legacy o que necesitan Chromium APIs

4.4 Otras utilidades offline-first

Tecnologia	Proposito	Peso	file://
Tesseract.js	OCR offline: extrae texto de imagenes escaneadas. Datos de idioma ~5-10 MB.	~50 KB (core)	Parcial
PDF.js	Renderiza PDFs en el navegador sin plugins. Ideal para visores de documentos.	~1.5 MB	Si
jsPDF	Genera PDFs desde JS. Facturas, reportes, certificados.	~500 KB	Si
SheetJS (XLSX)	Lee y escribe Excel. Importacion/exportacion de datos tabulares.	~350 KB	Si
ApexCharts	Graficos interactivos: barras, lineas, tortas, radar, heatmap.	~250 KB	Si

5. Mini IA Local — Arquitectura

El componente mas diferencial del stack: un sistema de IA que corre 100% local, sin conexion a internet, capaz de leer la base de datos del usuario, buscar en su documentacion y responder preguntas en lenguaje natural.

5.1 Niveles de inteligencia

Nivel 1: Busqueda inteligente (sin ML)

Usa FlexSearch o MiniSearch para busqueda full-text con scoring BM25/TF-IDF. Autocomplete, busqueda difusa tolerante a errores, sugerencias mientras escribes. Ideal para catalogos, historial de transacciones y documentacion tecnica. Cero descarga de modelos, instantaneo, peso ~7 KB.

Nivel 2: Busqueda semantica con embeddings

Usa Transformers.js con modelos multilinguees para convertir texto en vectores (embeddings) y buscar por significado, no por palabras exactas. Modelo recomendado: *Xenova/all-MiniLM-L6-v2* (80 MB cuantizado, soporta espanol). Los embeddings se almacenan en Dexie/SQLite y se comparan con similitud coseno. Primera descarga unica de ~80 MB, luego funciona 100% offline.

Nivel 3: Preguntas y respuestas (QA extractivo)

Usa Transformers.js con un modelo QA multilinguee (distilled BERT, ~70 MB) que extrae la respuesta directamente de un fragmento de texto. Combinado con los niveles 1 y 2 forma un pipeline RAG (Retrieval Augmented Generation) completo sin servidor.

5.2 Pipeline RAG local

El usuario hace una pregunta -> el sistema busca fragmentos relevantes en la BD y documentacion -> extrae la respuesta del mejor fragmento. Todo corre en el navegador o en el .exe, sin enviar datos a ningun servidor.

Pas o	Accion	Tecnologia	Tiempo
1	El usuario escribe una pregunta en lenguaje natural	Input field Alpine.js + FlexSearch autocomplete	Instantaneo
2	FlexSearch recupera top-10 fragmentos de BD y documentacion por keywords	FlexSearch (indice en memoria + IndexedDB)	< 5 ms
3	Transformers.js genera embeddings de pregunta y fragmentos -> similitud coseno -> re-rank top-5	Transformers.js (all-MiniLM-L6-v2)	~200 ms
4	Modelo QA extrae la respuesta del mejor fragmento	Transformers.js (distilbert multilingual)	~300-500 ms
5	Muestra la respuesta con fragmento de referencia (citation)	Alpine.js render en UI	Instantaneo

5.3 Predicciones desde la BD

No todo requiere redes neuronales. Para predicciones practicas desde datos locales:

- **Series temporales:** Promedio movil simple, suavizado exponencial, tendencia lineal. Implementacion en ~20 lineas de JS. Ideal para pronosticos de ventas, inventario, flujo de caja.
- **Clasificacion basica:** K-Nearest Neighbors (KNN) implementado en JS puro. Compara el caso actual con los N casos mas similares en BD. Ideal para diagnosticos, recomendaciones de productos, categorizacion automatica.
- **Reglas de negocio** via sistema experto simple: arboles de decision codificados desde la configuracion del usuario. Sin entrenamiento, sin modelos.
- **Clustering:** K-Means en JS puro para segmentacion de clientes o productos. Agrupa automaticamente por patrones de uso o compra.
- **Regresion lineal:** Minimimos cuadrados ordinarios en ~15 lineas de JS. Predice valores numericos a partir de datos historicos.

5.4 Stack tecnologico de la Mini IA

Componente	Tecnologia	Peso	Notas
Runtime ML	Transformers.js v4	~9 MB	Apache-2.0. ONNX WASM backend.
Embeddings	all-MiniLM-L6-v2	~80 MB	Soporta espanol. Cuantizado (Q8).
QA Extractivo	distilbert multilingual	~70 MB	Modelo distilled BERT. Buen balance calidad/velocidad.
Busqueda	FlexSearch o MiniSearch	~7-16 KB	Indice full-text persistente en IndexedDB.
Worker	Web Worker (navegador) / Bun Worker (.exe)	—	Obligatorio: no bloquear UI con inferencia.
Almacenamiento	Dexie + IndexedDB	—	Embeddings e indices guardados en DB local.

A tener en cuenta

Los modelos ONNX se descargan UNA VEZ (primera ejecucion) y quedan cacheados. La descarga inicial requiere internet (~150 MB total). Despues funciona 100% offline. Alternativa: incluir modelos en el .exe (aumenta el peso final pero elimina la descarga inicial).

6. Estrategia de Venta y Entrega

El objetivo final es entregar un producto que el cliente perciba como un software profesional, no como una pagina web. La percepcion del formato de entrega impacta directamente en el precio que puedes cobrar.

6.1 Percepcion vs Realidad

Un archivo .html que se abre en el navegador es funcionalmente identico a un .exe. Pero el cliente ve el .html como 'algo que me enviaron por correo' y el .exe como 'un programa que instale'. La diferencia es psicologica, no tecnica.

Formato	Percepcion del cliente	Rango de precio tipico	Proteccion
.html suelto	Es un archivo cualquiera	\$ 50 - \$ 200	Nula (codigo fuente visible)
.exe (Bun)	Es un programa instalado	\$ 500 - \$ 2.000	Media (binario ofuscado)
.exe (Tauri)	Es una app ligera profesional	\$ 800 - \$ 3.000	Alta (binario nativo)

6.2 Personalizacion profesional

- **Icono de aplicacion:** .ico (Windows) integrado en el .exe. El cliente ve tu logo en el escritorio, no el icono generico de Bun/node.
- **Ventana sin bordes de navegador:** Sin barra de direcciones, sin botones de navegador. Parece una app nativa. Tauri y Neutralino lo hacen por defecto. Con Bun --compile puedes usar un WebView personalizado.
- **Splash screen:** Pantalla de carga con logo de la empresa mientras se inicia la app. Pequeno detalle que marca la diferencia en percepcion profesional.
- **Nombre del proceso:** En el administrador de tareas aparece como 'MiApp.exe' no como 'node.exe' o 'bun.exe'.
- **Instalador opcional:** NSIS o Inno Setup para crear un setup.exe que instale en Program Files, cree acceso directo y se desinstale desde el Panel de Control.

6.3 Canales de entrega

Entrega inicial: Enlace de descarga cifrada (Mega, Google Drive con contrasena). El cliente recibe un ejecutable portable que funciona sin instalacion.

Actualizaciones: Mecanismo simple: la app verifica un archivo JSON local o usando el sistema de archivos para detectar version nueva. O descarga manual: 'descargue la nueva version desde el enlace que le envie'.

Licenciamiento: Sin DRM complejo. Opciones practicas: archivo de licencia .key generado con tu firma, o codigo de activacion offline validado con hash.

7. Tabla Comparativa Final

Las 5 opciones de empaquetado lado a lado para elegir segun el perfil del proyecto:

Opcion	Peso .exe	Runtime	Prot. IP	Curva	Ideal para
.html puro	0 MB + assets	Navegador	Nula	Nula	Demos rapidas, entregas internas, presupuesto minimo
Bun --compile	50-55 MB	Bun incluido	Ofuscado	JS puro	Apps complejas con backend local, BD SQLite, API propia
Neutralino.js	1-2 MB	WebView OS	Binario	Config basica	Apps ligeras, wrappers de UI, kioskos
Tauri	5-15 MB	WebView + Rust	Binario++	Rust basico	Apps medianas, rendimiento critico, actualizaciones OTA
Electron	150-300 MB	Chromium	Ofuscado	JS puro	Apps legacy, acceso completo a APIs nativas

Recomendacion general:

Para el 80% de los proyectos freelance, la combinacion optima es: **desarrollo en modo Lite (file:// + Dexie)** con entrega final compilada via **Bun --compile**. Esto da el mejor balance entre simplicidad de desarrollo (sin build loop) y profesionalismo en la entrega (.exe con peso aceptable de ~55 MB). Si el peso es critico (menos de 10 MB), evaluar Neutralino.js o Tauri.

8. Conclusion

El Stack Offline-First Profesional no es solo una coleccion de tecnologias: es una estrategia de negocio. Permite al desarrollador freelance LatAm:

- Crear software que funciona 100% sin internet, ideal para clientes con conectividad limitada.
- Entregar productos con percepcion profesional (.exe con logo, splash, nombre de proceso).
- Proteger el codigo fuente del cliente (binario compilado en lugar de HTML visible).
- Ofrecer IA local sin enviar datos del cliente a ningun servidor externo.
- Mantener la simplicidad del stack original (Alpine.js, Dexie, DaisyUI) con la opcion de compilar a ejecutable cuando el proyecto lo requiera.
- Cobrar precios acordes al valor percibido (\$500-\$3000 vs \$50-\$200).

Proximos pasos

1. Definir spec del proyecto con spec-creator
2. Generar codigo base con code-generator (modo Lite por defecto)
3. Desarrollar modulos uno por uno
4. Compilar a .exe para entrega al cliente
5. Validar con Playwright + compliance guard